

Базы данных
Лабораторная работа №4
Вариант №854032

Выполнила:
Исаева Александра-Ирина Антоновна

Проверила:
Воронина Дарья Сергеевна

Содержание

1/ Текст задания.....	3
2/ Реализация запросов на SQL.....	4
2.1 Запрос 1.....	4
2.2 Запрос 2.....	6
3/ Вывод.....	7

1/ Текст задания

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор.

Изменяются ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ.

Вывести атрибуты: Н_ЛЮДИ.ИД, Н_ВЕДОМОСТИ.ДАТА.

Фильтры (AND):

- Н_ЛЮДИ.ИМЯ < Владимир.
- Н_ВЕДОМОСТИ.ЧЛВК_ИД > 142390.
- Н_ВЕДОМОСТИ.ЧЛВК_ИД > 117219.

Вид соединения: LEFT JOIN.

2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ОТЧЕСТВО, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.НАЧАЛО.

Фильтры: (AND)

- Н_ЛЮДИ.ИМЯ = Владимир.
- Н_ОБУЧЕНИЯ.ЧЛВК_ИД < 112514.

Вид соединения: INNER JOIN.

2/ Реализация запросов на SQL

2.1 Запрос 1

```
SELECT Н_ЛЮДИ.ИД, Н_ВЕДОМОСТИ.ДАТА
FROM Н_ЛЮДИ
LEFT JOIN Н_ВЕДОМОСТИ ON Н_ВЕДОМОСТИ.ЧЛВК_ИД = Н_ЛЮДИ.ИД
WHERE Н_ЛЮДИ.ИМЯ < 'Владимир' AND
      Н_ВЕДОМОСТИ.ЧЛВК_ИД > 142390 AND
      Н_ВЕДОМОСТИ.ЧЛВК_ИД > 117219;
```

Так как условие `Н_ВЕДОМОСТИ.ЧЛВК_ИД > 117219` выполняется всегда при выполнении `Н_ВЕДОМОСТИ.ЧЛВК_ИД > 142390`, будем рассматривать запрос без него.

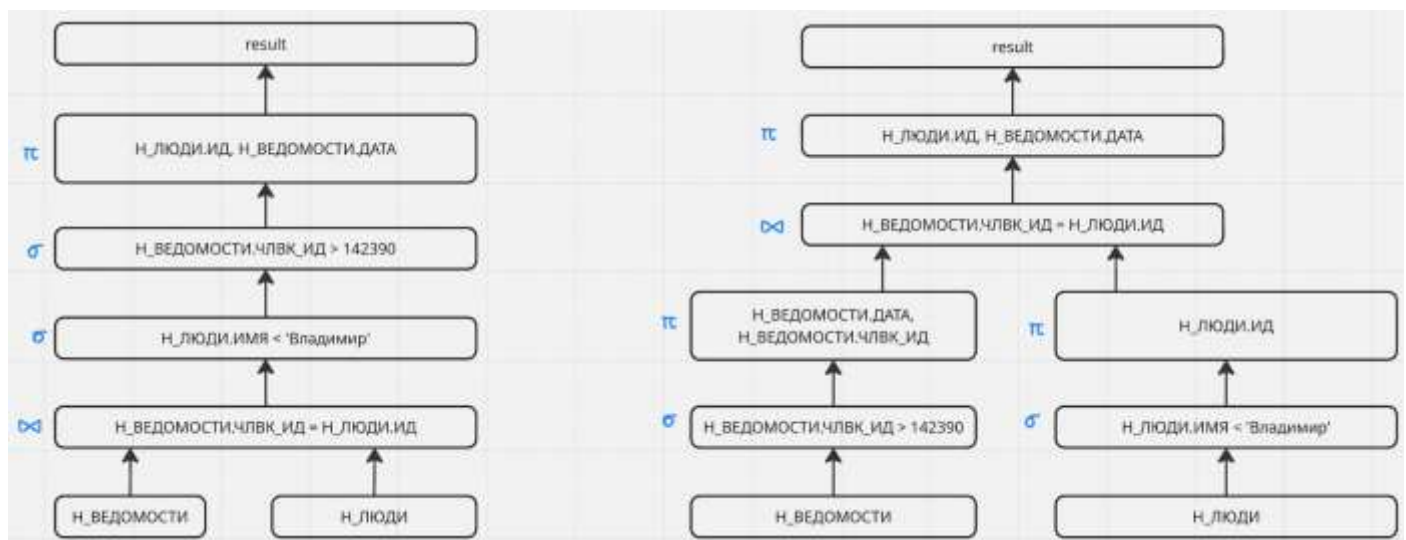
Индексы:

1. Для сущности `Н_ЛЮДИ` индекс на атрибуте `ИМЯ` (B-tree, так как выборка происходит с использованием операторов сравнения, оптимально использование именно B-tree). Это ускорит фильтрацию строк с условием `Н_ЛЮДИ.ИМЯ < 'Владимир'`.

```
CREATE INDEX ON Н_ЛЮДИ USING BTREE(ИМЯ);
```

2. Для сущности `Н_ВЕДОМОСТИ` индекс на атрибуте `ЧЛВК_ИД` (также B-tree, так как выборка происходит с использованием операторов сравнения). Это ускорит фильтрацию строк с условием `Н_ВЕДОМОСТИ.ЧЛВК_ИД > 142390`.

```
CREATE INDEX ON Н_ВЕДОМОСТИ USING BTREE(ЧЛВК_ИД);
```



Второй (правый) план является оптимальным, так как выборка происходит на более ранних этапах, идет соединение только нужных атрибутов, и, как следствие, размер промежуточных данных меньше. Вместо полного сканирования будет использоваться индексный скан.

```
EXPLAIN ANALYSE SELECT Н_ЛЮДИ.ИД, Н_ВЕДОМОСТИ.ДАТА
FROM Н_ЛЮДИ
LEFT JOIN Н_ВЕДОМОСТИ ON Н_ВЕДОМОСТИ.ЧЛВК_ИД = Н_ЛЮДИ.ИД
```

```

WHERE Н_ЛЮДИ.ИМЯ < 'Владимир' AND
      Н_ВЕДОМОСТИ.ЧЛВК_ИД > 142390;

Hash Join (cost=697.38..5486.61 rows=15613 width=12) (actual time=5.351..22.792
rows=15223 loops=1)
  Hash Cond: ("Н_ВЕДОМОСТИ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
    -> Bitmap Heap Scan on "Н_ВЕДОМОСТИ" (cost=511.35..5181.58 rows=45298 width=12)
(actual time=2.208..11.710 rows=45630 loops=1)
      Recheck Cond: ("ЧЛВК_ИД" > 142390)
      Heap Blocks: exact=1652
        -> Bitmap Index Scan on "ВЕД_ЧЛВК_FK_IFK" (cost=0.00..500.03 rows=45298
width=0) (actual time=2.007..2.008 rows=45630 loops=1)
          Index Cond: ("ЧЛВК_ИД" > 142390)
        -> Hash (cost=163.97..163.97 rows=1764 width=4) (actual time=3.126..3.127
rows=1765 loops=1)
          Buckets: 2048 Batches: 1 Memory Usage: 79kB
          -> Seq Scan on "Н_ЛЮДИ" (cost=0.00..163.97 rows=1764 width=4) (actual
time=0.014..2.821 rows=1765 loops=1)
            Filter: (("ИМЯ")::text < 'Владимир')::text)
            Rows Removed by Filter: 3353
Planning Time: 0.533 ms
Execution Time: 23.559 ms

```

Запрос объединяет две таблицы ("Н_ВЕДОМОСТИ" и "Н_ЛЮДИ") с использованием хеш-соединения.

Первая часть соединения (строится хеш-таблица):

1. Сначала выполняется полное сканирование таблицы "Н_ЛЮДИ" с фильтром, который оставляет только людей с именами, меньшими "Владимир".
2. Из 5118 строк (1765 + 3353) остаётся 1765 строк.
3. Эти строки используются для построения хеш-таблицы в памяти (79 КБ).

Вторая часть соединения (проверяется по хеш-таблице):

1. Используется индекс "ВЕД_ЧЛВК_FK_IFK" для быстрого нахождения строк в "Н_ВЕДОМОСТИ" с "ЧЛВК_ИД" > 142390.
2. Найдено 45630 строк, которые затем соединяются с хеш-таблицей.

Результат:

- Получено 15223 строк, соответствующих условиям соединения.
- Весь запрос выполнен за 23.559 мс, из которых 0.533 мс ушло на планирование.
- Оптимизатор выбрал хеш-соединение, так как одна из таблиц ("Н_ЛЮДИ" после фильтрации) достаточно мала (1765 строк), чтобы эффективно поместиться в хеш-таблицу в памяти. Для большой таблицы "Н_ВЕДОМОСТИ" используется индексный доступ, что ускоряет обработку.

2.2 Запрос 2

```
SELECT Н_ЛЮДИ.ОТЧЕСТВО, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.НАЧАЛО
FROM Н_ЛЮДИ
INNER JOIN Н_ОБУЧЕНИЯ ON Н_ОБУЧЕНИЯ.ЧЛВК_ИД = Н_ЛЮДИ.ИД
INNER JOIN Н_УЧЕНИКИ ON Н_ОБУЧЕНИЯ.ЧЛВК_ИД = Н_УЧЕНИКИ.ЧЛВК_ИД
WHERE (Н_ЛЮДИ.ИМЯ > 'Владимир' AND Н_ОБУЧЕНИЯ.ЧЛВК_ИД < 112514);
```

Индексы:

1. Для Н_ЛЮДИ.ИД индекс не нужен, этот атрибут является первичным ключом (индекс B-tree создаётся по умолчанию).

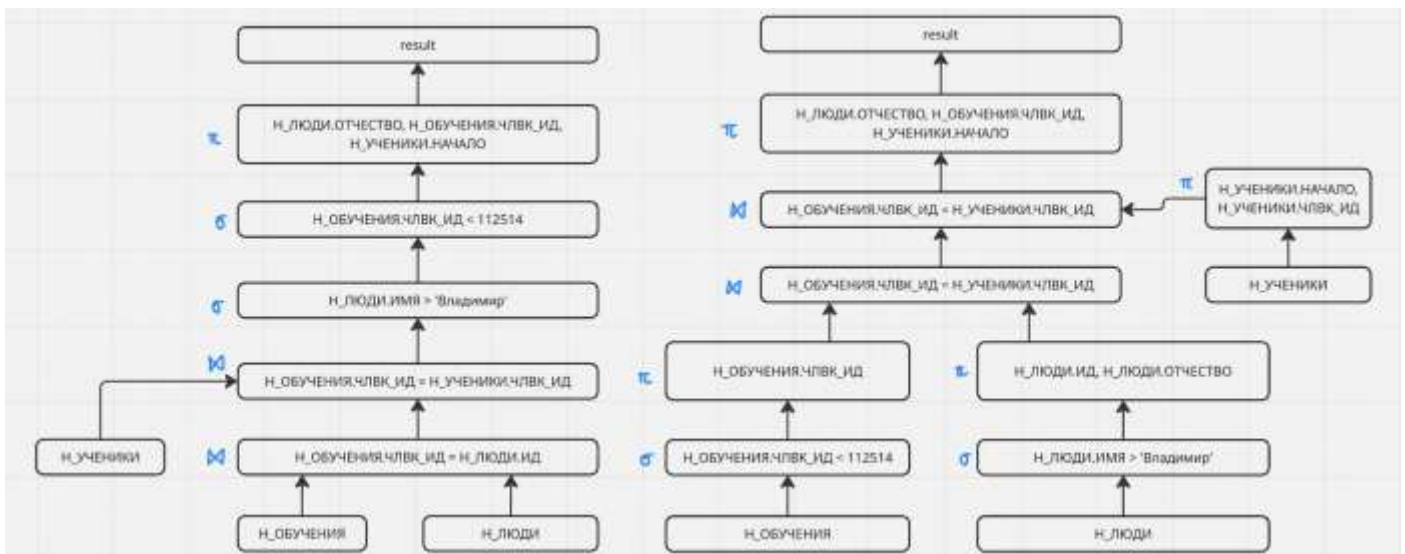
2. Индекс для внешнего ключа Н_ОБУЧЕНИЯ.ЧЛВК_ИД (B-tree) оптимизирует соединение таблиц Н_ОБУЧЕНИЯ и Н_ЛЮДИ.

```
CREATE INDEX ON Н_ОБУЧЕНИЯ USING BTREE (Н_ОБУЧЕНИЯ.ЧЛВК_ИД);
```

3. Индекс для внешнего ключа Н_УЧЕНИКИ.ЧЛВК_ИД (B-tree) оптимизирует соединение таблиц Н_ОБУЧЕНИЯ и Н_УЧЕНИКИ.

```
CREATE INDEX ON Н_УЧЕНИКИ USING BTREE (Н_УЧЕНИКИ.ЧЛВК_ИД);
```

4. Для сущности Н_ЛЮДИ индекс на атрибуте ИМЯ (также как в запросе 1)



Второй (правый) план является оптимальным, так как выборка происходит на более ранних этапах, идет соединение только нужных атрибутов, и, как следствие, размер промежуточных данных меньше. Вместо полного сканирования будет использоваться индексный скан.

Второй (правый) план является оптимальным, из-за того, что выборка происходит на более ранних этапах, идет соединение только нужных атрибутов, и, как следствие, размер промежуточных данных меньше. При добавлении индексов планы выполнения запросов изменится, вместо полного сканирования будет использоваться индексный скан, Nested Loops Join будет работать быстрее.

```
EXPLAIN ANALYSE SELECT Н_ЛЮДИ.ОТЧЕСТВО, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.НАЧАЛО
FROM Н_ЛЮДИ
INNER JOIN Н_ОБУЧЕНИЯ ON Н_ОБУЧЕНИЯ.ЧЛВК_ИД = Н_ЛЮДИ.ИД
INNER JOIN Н_УЧЕНИКИ ON Н_ОБУЧЕНИЯ.ЧЛВК_ИД = Н_УЧЕНИКИ.ЧЛВК_ИД
WHERE (Н_ЛЮДИ.ИМЯ > 'Владимир' AND Н_ОБУЧЕНИЯ.ЧЛВК_ИД < 112514);
```

```
Nested Loop (cost=12.94..405.66 rows=832 width=32) (actual time=0.165..4.323 rows=566
loops=1)
  Join Filter: ("Н_ОБУЧЕНИЯ"."ЧЛВК_ИД" = "Н_УЧЕНИКИ"."ЧЛВК_ИД")
    -> Hash Join (cost=12.65..194.61 rows=177 width=28) (actual time=0.140..3.524
rows=162 loops=1)
      Hash Cond: ("Н_ЛЮДИ"."ИД" = "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД")
      -> Seq Scan on "Н_ЛЮДИ" (cost=0.00..163.97 rows=3243 width=24) (actual
time=0.013..2.989 rows=3243 loops=1)
        Filter: (("ИМЯ")::text > 'Владимир'::text)
        Rows Removed by Filter: 1875
      -> Hash (cost=9.17..9.17 rows=279 width=4) (actual time=0.085..0.085
rows=264 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 18kB
        -> Index Only Scan using "ОБУЧ_ЧЛВК_FK_I" on "Н_ОБУЧЕНИЯ"
(cost=0.28..9.17 rows=279 width=4) (actual time=0.013..0.039 rows=264 loops=1)
          Index Cond: ("ЧЛВК_ИД" < 112514)
          Heap Fetches: 0
        -> Index Scan using "УЧЕН_ОБУЧ_FK_I" on "Н_УЧЕНИКИ" (cost=0.29..1.13 rows=5
width=12) (actual time=0.002..0.004 rows=3 loops=162)
          Index Cond: ("ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
    Planning Time: 1.983 ms
    Execution Time: 4.440 ms
```

Запрос соединяет три таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ и Н_УЧЕНИКИ. Сначала выполняется хэш-соединение Н_ЛЮДИ и Н_ОБУЧЕНИЯ. Затем результат соединяется с Н_УЧЕНИКИ.

Последовательность:

1. Сначала PostgreSQL фильтрует таблицу Н_ЛЮДИ, оставляя только людей с именами больше "Владимир" (3243 строки)
2. Затем выбирает записи из Н_ОБУЧЕНИЯ с ЧЛВК_ИД < 112514 (264 строки) через индекс
3. Соединяет эти две таблицы по условию Н_ЛЮДИ.ИД = Н_ОБУЧЕНИЯ.ЧЛВК_ИД (162 строки)
4. Для каждой из этих 162 строк ищет соответствующие записи в Н_УЧЕНИКИ через индекс.

Возвращается 566 строк.

3/ Вывод

При выполнении лабораторной работы я познакомилась с использованием индексов для ускорения обработки запросов в SQL, с планами выполнения запросов, их построением и анализом.